ADVANCED PROGRAMMING LANGUAGES
Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Designing Datatypes

In the previous lectures we have studied data types (tuples, records, variants, recursive types) and higher-order functions as separate topics.

In practice, however, a data type is never just a collection of values: rather, it is a data abstraction, a set of values together with the operations that create, inspect, and transform them.

- A `Shape` is not just `Circle` and/or `Rectangle`; it is circles and rectangles together with `area`, `perimeter`, `scale`, and whatever other operations make shapes useful.
- A `List` is not just `[]` and `(:)`; it is the constructors together with `map`, `filter`, `foldr`, and the rest.

The fundamental design question is: *how do we organize the relationship between the values and their operations?*

Different languages answer this differently, and the choice has real consequences for how easy it is to extend and maintain the code.

The design question

Consider the familiar `Shape` example with three operations — `area`, `perimeter`, and `scale`:

We have a type with two variants (`Circle`, `Rectangle`) and three operations (`area`, `perimeter`, `scale`). The code can be organized in two fundamentally different ways:

- **Functional design**: the data definition and the operations are separate. The data type defines the structure; the operations are standalone functions that receive the data as an argument and dispatch on its shape.
- **Object-oriented design**: the data definition and the operations are bundled together. Each piece of data carries its own implementations of the operations, and the dispatch happens internally when a method is invoked.

The distinction is not merely stylistic — it affects what is easy to extend and what requires modifying existing code.

Functional design: data + external functions

In this style, the data type is defined in one place and operations are standalone functions that pattern-match on the constructors. The data is *inert* — it carries no behaviour.

Haskell: modules

In Haskell, the natural organizational unit is the **module**. A module groups a data type declaration with the functions that operate on it:

```haskell
module Shapes (Shape(..), area, perimeter, scale) where

data Shape = Circle Double
           | Rectangle Double Double

area :: Shape -> Double
area (Circle r)      = pi * r ^ 2
area (Rectangle b h) = b * h

perimeter :: Shape -> Double
perimeter (Circle r)      = 2 * pi * r
perimeter (Rectangle b h) = 2 * (b + h)

scale :: Double -> Shape -> Shape
scale f (Circle r)      = Circle (r * f)
scale f (Rectangle b h) = Rectangle (b * f) (h * f)
```

The module header `module Shapes (Shape(..), area, perimeter, scale) where` declares what is exported: the type `Shape` with all its constructors (`..`), and the three functions. Clients import the module and use the exported names:

```haskell
import Shapes

myCircle = Circle 3.0
a = area myCircle         -- 28.27...
p = perimeter myCircle    -- 18.84...
```

Several features of this design are worth noting:

- The functions are **standalone**: they are not "owned by" the data type. They are ordinary values in the module namespace, and they can be imported selectively, renamed, or passed as arguments to higher-order functions.
- The coupling between data and operations is **by convention and by type**: the functions refer to `Shape` in their signatures, and the module groups them together, but there is no linguistic mechanism that ties `area` to `Shape` more tightly than any other function.
- **Adding a new operation** (e.g. `describe :: Shape -> String`) requires only writing a new function — no existing code is modified.
- **Adding a new variant** (e.g. `Triangle Double Double Double`) requires modifying *every* existing function to add a clause for the new constructor. The compiler helps: GHC's exhaustiveness checker will warn about missing cases.

Elixir: modules

Elixir follows the same functional design, with modules as the organizational unit:

```
defmodule Shapes do
  def area({:circle, r}),      do: :math.pi() * r ** 2
  def area({:rectangle, b, h}), do: b * h

  def perimeter({:circle, r}),      do: 2 * :math.pi() * r
  def perimeter({:rectangle, b, h}), do: 2 * (b + h)

  def scale(f, {:circle, r}),      do: {:circle, r * f}
  def scale(f, {:rectangle, b, h}), do: {:rectangle, b * f, h * f}
end
```

The structure is close to Haskell: data is defined by convention (tagged tuples), operations are functions in a module, and each function dispatches on the tag via pattern matching. We may replace tagget tuples with structs to get even closer to Haskell, as shown below:

```elixir
defmodule Circle do
  defstruct [:radius]
end

defmodule Rectangle do
  defstruct [:base, :height]
end

defmodule Shapes do
  def area(%Circle{radius: r}),              do: :math.pi() * r ** 2
  def area(%Rectangle{base: b, height: h}),  do: b * h

  def perimeter(%Circle{radius: r}),              do: 2 * :math.pi() * r
  def perimeter(%Rectangle{base: b, height: h}),  do: 2 * (b + h)

  def scale(f, %Circle{radius: r}),              do: %Circle{radius: r * f}
  def scale(f, %Rectangle{base: b, height: h}),  do: %Rectangle{base: b * f, height: h * f}
end
```

Irrespective of whether we use structs or not, the trade-offs are the same — easy to add operations, hard to add variants. One difference from Haskell: Elixir has no exhaustiveness checker, so adding a new variant without updating all functions produces a runtime error (`FunctionClauseError`) rather than a compile-time warning.

Scala: the functional style

Scala can express the same functional design using `enum` declarations with external functions:

```scala
package FunShapes

enum Shape:
  case Circle(radius: Double)
  case Rectangle(base: Double, height: Double)

import Shape.*

def area(s: Shape): Double = s match
  case Circle(r)      => Math.PI * Math.pow(r, 2)
  case Rectangle(b,h) => b * h

def perimeter(s: Shape): Double = s match
  case Circle(r)      => 2 * Math.PI * r
  case Rectangle(b,h) => 2 * (b + h)

def scale(f: Double, s: Shape): Shape = s match
  case Circle(r)      => Circle(r * f)
  case Rectangle(b,h) => Rectangle(b * f, h * f)
```

The `enum` declaration introduces the type `Shape` and its two data constructors `Circle` and `Rectangle`. The functions are top-level definitions in the package, not methods on `Shape` — they dispatch on the variant via `match`, just as in Haskell and Elixir. The `enum` is syntactic sugar that concisely defines a closed set of variants; under the hood, it generates a `sealed trait Shape` with `case class Circle(...)` and `case class Rectangle(...)` as subtypes. This desugaring will become important when we move to the OO style.

Object-oriented design: data + internal methods

In this style, each variant carries its own implementations of the operations. The operations are declared in a common interface (a trait, an abstract class, a protocol) and each variant provides its own definition.

Scala: from enums to traits and case classes

In the functional style above, we used Scala's `enum` to define `Shape`. Can we add methods directly to the enum? We can — but only methods with a *shared* implementation:

```
enum Shape:
  case Circle(radius: Double)
  case Rectangle(base: Double, height: Double)

  def describe: String = this.toString   // shared by all cases — OK
```

What we *cannot* do with `enum` is give each case its own *distinct* implementation of a method. There is no syntax to write "when I'm a `Circle`, compute area this way; when I'm a `Rectangle`, compute it that way" inside the enum — other than pattern matching on `this`, which is just the functional style with extra steps.

To achieve the true OO design — where each variant carries its own behaviour — we need to desugar the enum into what it really is: a **sealed trait** with **case classes**, each of which can define its own method bodies.

```
package OOShapes

sealed trait Shape:
  def area: Double
  def perimeter: Double
  def scale(f: Double): Shape

case class Circle(radius: Double) extends Shape:
  def area: Double = Math.PI * Math.pow(radius, 2)
  def perimeter: Double = 2 * Math.PI * radius
  def scale(f: Double): Shape = Circle(radius * f)

case class Rectangle(base: Double, height: Double) extends Shape:
  def area: Double = base * height
  def perimeter: Double = 2 * (base + height)
  def scale(f: Double): Shape = Rectangle(base * f, height * f)
```

Now the operations are **methods** declared in the trait and implemented in each case class. A client calls them with dot notation:

```
val c = Circle(3.0)
c.area          // 28.27...
c.perimeter     // 18.84...
c.scale(2.0)    // Circle(6.0)
```

The trade-offs are reversed compared to the functional style:

- **Adding a new variant** (e.g. `Triangle`) requires only defining a new case class that extends `Shape` and implements all the methods — no existing code is modified.
- **Adding a new operation** (e.g. `describe: String`) requires modifying the trait *and* every existing case class.

The key insight is that `enum` and `sealed trait + case classes` are not two unrelated mechanisms — the enum *is* the trait hierarchy, just in a more concise notation. The conciseness comes at a cost: enums lock you into the functional design. When you need per-variant method implementations — the OO design — you must work with the desugared form explicitly.

Methods vs functions

The distinction between `area(s)` (functional) and `s.area` (OO) is not just syntactic. It reflects where the *dispatch decision* lives:

- In the functional style, the function `area` owns the dispatch: it pattern-matches on the constructor and decides what to do. The data is passive.
- In the OO style, the data owns the dispatch: calling `s.area` invokes the `area` method that the specific variant (`Circle` or `Rectangle`) provides. The function name is shared, but the implementation is distributed across the variants.

In Haskell and Elixir, only the functional style is available — there are no methods, no dot notation, no dynamic dispatch on data. In Scala, both styles coexist, and the choice is a design decision.

Haskell and Elixir: no OO design?

Haskell and Elixir lack classes and methods in the OO sense, so the OO design as shown in Scala is not directly available. However, both languages provide mechanisms that partially address the same concern:

- **Haskell type classes** define a set of operations (methods) that can be implemented differently for different types. This is not the same as the OO design (the operations are not "inside" the data), but it provides a way to add behaviour to existing types without modifying them. We will study type classes in detail in a later lecture.
- **Elixir protocols** serve a similar purpose: they define a set of functions that can be implemented for different data types (structs). For instance, the `String.Chars` protocol defines how a type can be converted to a string, and each struct can provide its own implementation. Again, this is not OO dispatch, but it enables a form of extensible behaviour.

These mechanisms offer a *third way* between the pure functional and the pure OO designs — one that we will explore further when we study type classes and ad-hoc polymorphism.

The Expression Problem

The tension between the two designs is known as the **Expression Problem** (a term coined by Philip Wadler in 1998). It can be stated concisely:

> **Can we extend both the data type (new variants) and the operations (new functions) without modifying existing code and without losing type safety?**

	Add a new operation	**Add a new variant**
Functional (Haskell, Elixir, Scala functional)	Easy: write a new function	Hard: modify all existing functions
OO (Scala OO, Java)	Hard: modify the trait and all classes	Easy: add a new class

Neither design solves the Expression Problem on its own. Advanced mechanisms — type classes in Haskell, extension methods and givens in Scala, protocols in Elixir — provide partial solutions, which we will study later in the course.

Summary

	Haskell	**Scala**	**Elixir**
Organizational unit	Module	Package / Object / Class	Module
Data definition	`data` declaration	`sealed trait + case class`	Structs / tagged tuples
Operations	Standalone functions	Methods (OO) or functions (functional)	Standalone functions
Dispatch mechanism	Pattern matching	Pattern matching (functional) or dynamic dispatch (OO)	Pattern matching
Adding operations	Easy (new function)	Easy in both styles	Easy (new function)

	Haskell	Scala	Elixir
Adding variants	Modify all functions	Easy (OO) / modify all functions (functional)	Modify all functions
Extensible dispatch	Type classes	Givens / extension methods	Protocols

Exercises

Exercise 1. Take the `Shape` example and add a third variant, `Triangle Double Double Double` (three sides). Implement it in both the functional style (Haskell or Elixir) and the OO style (Scala). Which modification is more invasive?

Exercise 2. Now add a new operation, `describe :: Shape -> String`, that produces a human-readable description of the shape. Again, implement it in both styles. Which modification is more invasive this time?

Exercise 3. Consider a simple expression language:

```
data Expr = Lit Int | Add Expr Expr | Mul Expr Expr
```

with two operations: `eval :: Expr -> Int` and `pretty :: Expr -> String`.

a. Implement this in Haskell using the functional design.

b. Implement the same in Scala using the OO design.

c. Now add a new variant `Neg Expr` (negation). Which implementation requires more changes?

d. Now add a new operation `depth :: Expr -> Int` (the depth of the expression tree). Which implementation requires more changes?

e. Summarize your observations in terms of the Expression Problem.

Exercise 4. In Elixir, define a protocol `Describable` with a single function `describe/1`. Implement it for two structs: `Circle` and `Rectangle`. Compare this with the Haskell type class approach (which we will formalize later) and with Scala's trait-based OO approach. What are the similarities and differences?

Exercise 5. Consider the following Haskell module:

```haskell
module Stack (Stack, empty, push, pop, top, isEmpty) where

newtype Stack a = MkStack [a]

empty :: Stack a
empty = MkStack []

push :: a -> Stack a -> Stack a
push x (MkStack xs) = MkStack (x : xs)

pop :: Stack a -> Stack a
pop (MkStack [])     = error "empty stack"
pop (MkStack (_:xs)) = MkStack xs

top :: Stack a -> a
top (MkStack [])    = error "empty stack"
top (MkStack (x:_)) = x

isEmpty :: Stack a -> Bool
isEmpty (MkStack xs) = null xs
```

a. Note that the export list includes `Stack` but *not* `MkStack`. What does this mean for clients of the module? Can they pattern-match on the internal representation?

b. A client can only interact with `Stack` values through the exported functions. This is a form of *data abstraction* — the representation is hidden behind an interface. How does this compare with the OO approach, where the representation is hidden inside an object?

c. Could the module author change the internal representation (e.g. from a list to a pair of lists for an efficient queue) without breaking any client code? Explain.

REFERENCES

The discussion on functional vs object-oriented design and the Expression Problem draws on P. Wadler, *The Expression Problem*, message to the Java-genericity mailing list, 1998, and on Chapter 19 of *Programming in Scala (5th Edition)*, M. Odersky, L. Spoon, B. Venners, F. Sommers, Artima Press, 2021.

The treatment of Haskell modules draws on Chapter 11 of *Programming in Haskell (2nd Edition)*, G. Hutton, Cambridge University Press, 2016.